

Trabalho — Ponteiros em C

Objetivo

Projetar e implementar um programa que utilize **ponteiros** para manipular variáveis, conforme os conceitos apresentados em aula.

O trabalho será realizado em duas etapas:

Etapla 1 — em casa: elaboração e entrega somente do pseudocódigo.

Etapla 2 — laboratório: implementação correspondente em C e apresentação individual ou em grupo.

O objetivo é compreender como uma função pode receber **endereços de variáveis** e, por meio de ponteiros, modificar diretamente seus valores.

Contexto

Deseja-se construir um pequeno **sistema de gerenciamento de valores inteiros**.

O programa receberá dois valores inteiros e realizará diferentes operações sobre eles.

O sistema deverá ser capaz de:

ler dois valores inteiros;

trocar os valores das duas variáveis utilizando ponteiros;

calcular a soma dos valores;

calcular o produto dos valores;

determinar o maior e o menor valor;

apresentar os resultados de forma organizada.

👉 Algumas funções deverão produzir seus resultados **modificando variáveis por meio de seus endereços**, em vez de simplesmente retornar um valor.

ETAPA 1 — Preparação em casa

Tarefa

Elaborar o algoritmo completo em **pseudocódigo**, decompondo o problema em subprogramas.

O programa deverá possuir obrigatoriamente:

PROCEDIMENTO trocar(...)

PROCEDIMENTO calcularSoma(...)

PROCEDIMENTO calcularProduto(...)

PROCEDIMENTO determinarMaiorMenor(...)

PROCEDIMENTO mostrarResultados(...)

um algoritmo principal.

⚠️ Nesta primeira etapa, ainda não deverá ser escrito código em C.

No pseudocódigo, represente claramente quando um parâmetro corresponde ao **endereço de uma variável**.



Elementos obrigatórios

1. Procedimento trocar

Recebe:

endereço da primeira variável inteira;

endereço da segunda variável inteira.

O procedimento deverá trocar os conteúdos das duas variáveis.

Exemplo:

Antes:

a = 5

b = 8

Depois:

a = 8

b = 5

👉 A troca deverá modificar diretamente as variáveis fornecidas ao procedimento.

2. Procedimento calcularSoma

Recebe:

dois valores inteiros;

o endereço de uma variável destinada a armazenar a soma.

Deverá calcular:

$soma \leftarrow a + b$

e armazenar o resultado na variável indicada pelo endereço recebido.

Exemplo:

a = 8

b = 5

Resultado:

soma = 13

👉 O resultado não deverá ser produzido com **retornar**.

Ele deverá ser armazenado através do endereço recebido.

3. Procedimento calcularProduto

Recebe:

dois valores inteiros;

o endereço de uma variável destinada a armazenar o produto.

Deverá calcular:

$\text{produto} \leftarrow a \times b$

e armazenar o resultado na variável indicada.

Exemplo:

$a = 8$

$b = 5$

Resultado:

$\text{produto} = 40$

4. Procedimento determinarMaiorMenor

Recebe:

dois valores inteiros;

o endereço de uma variável destinada ao maior valor;

o endereço de uma variável destinada ao menor valor.

O procedimento deverá determinar simultaneamente:

maior

e

menor

e armazenar os resultados nas respectivas variáveis.

Exemplo:

$a = 8$

$b = 5$

Resultado:

$\text{maior} = 8$

$\text{menor} = 5$

👉 Este procedimento produz **dois resultados** através de parâmetros que representam endereços.

5. Procedimento mostrarResultados

Recebe:

os dois valores após a troca;

soma;

produto;

maior;

menor.

Deverá apresentar os resultados de forma organizada.

Exemplo:

Valor A após a troca: 8

Valor B após a troca: 5

Soma: 13

Produto: 40

Maior: 8

Menor: 5

👉 Este procedimento apenas apresenta os resultados.

Ele não deverá modificar as variáveis recebidas.



Algoritmo principal

O algoritmo principal deverá:

ler dois valores inteiros a e b;

trocar os valores de a e b utilizando o procedimento trocar;

calcular a soma dos valores utilizando calcularSoma;

calcular o produto utilizando calcularProduto;

determinar o maior e o menor valor utilizando determinarMaiorMenor;

apresentar os resultados utilizando mostrarResultados.

O algoritmo principal deverá essencialmente **coordenar as chamadas aos subprogramas**.

👉 Os cálculos não deverão ser repetidos dentro do algoritmo principal.

Restrições da Etapa 1

- ✗ Não entregar código C.
 - ✗ Não realizar todos os cálculos diretamente no algoritmo principal.
 - ✗ Não repetir no algoritmo principal operações que pertencem aos subprogramas.
 - ✗ Não utilizar vetores.
 - ✗ Não utilizar alocação dinâmica de memória.
 - ✗ Não utilizar ponteiros para ponteiros.
 - ✓ Utilizar procedimentos e parâmetros.
 - ✓ Representar claramente quais parâmetros correspondem a **endereços**.
 - ✓ Diferenciar valores de endereços.
 - ✓ O pseudocódigo deve permitir sua tradução direta para C.
-

Cenários de referência

Cenário 1

Entrada:

$a = 5$

$b = 8$

Após a troca:

$a = 8$

$b = 5$

Resultados:

Soma = 13

Produto = 40

Maior = 8

Menor = 5

Cenário 2

Entrada:

$a = 10$

$b = 3$

Após a troca:

$a = 3$

$b = 10$

Resultados:

Soma = 13

Produto = 30

Maior = 10

Menor = 3

Cenário 3

Entrada:

$a = -4$

$b = 6$

Após a troca:

$a = 6$

$b = -4$

Resultados:

Soma = 2

Produto = -24

Maior = 6

Menor = -4

Cenário 4

Entrada:

$a = 7$

$b = 7$

Após a troca:

$a = 7$

$b = 7$

Resultados:

Soma = 14

Produto = 49

Maior = 7

Menor = 7



Entrega — Etapa 1

Entregar **somente o pseudocódigo**, contendo:

algoritmo principal;
trocar;
calcularSoma;
calcularProduto;
determinarMaiorMenor;
mostrarResultados.



Objetivo desta primeira entrega: verificar se o aluno compreendeu a diferença entre **passar um valor e fornecer o endereço de uma variável**, permitindo que um subprograma modifique diretamente variáveis externas.



ETAPA 2 — Laboratório

A partir do pseudocódigo entregue, implementar o programa em **C**.

A implementação deverá utilizar obrigatoriamente os seguintes protótipos:

```
void trocar(int *a, int *b);  
void calcularSoma(int a, int b, int *soma);  
void calcularProduto(int a, int b, int *produto);  
void determinarMaiorMenor(int a, int b, int *maior, int *menor);  
void mostrarResultados(int a, int b, int soma, int produto, int maior, int menor);
```



Os protótipos deverão aparecer **antes de main()**.



As definições das funções deverão aparecer **depois de main()**.

Essa organização mantém exatamente o padrão exigido no laboratório anterior, no qual os protótipos ficavam antes de **main()** e as definições depois.

Trabalho — Funções e Subprogramas em C.pdf



Estrutura esperada

```
#include <stdio.h>
/* protótipos */
void trocar(int *a, int *b);
void calcularSoma(int a, int b, int *soma);
void calcularProduto(int a, int b, int *produto);
void determinarMaiorMenor(int a, int b, int *maior, int *menor);
void mostrarResultados(int a, int b, int soma, int produto, int maior, int menor);
int main(void) {
    /* declaração das variáveis */
    /* leitura */
    /* chamadas às funções */
    return 0;
}
/* definições das funções */
```

👉 **main()** deverá essencialmente **coordenar as chamadas às funções**.

👉 A lógica interna de cada operação deverá permanecer na função correspondente.



Uso dos ponteiros

Durante a implementação, observe particularmente a diferença entre:

a

👉 valor armazenado em **a**

&a

👉 endereço de **a**

p

👉 endereço armazenado no ponteiro **p**

***p**

👉 conteúdo da variável apontada por **p**

Essas quatro relações constituem a base conceitual da aula de Ponteiros.

PonteirosC.pdf

Por exemplo, uma chamada poderá assumir a forma:

```
trocar(&a, &b);
```

Nesse caso, **trocar** recebe os **endereços** das duas variáveis e poderá modificar seus conteúdos.



Apresentação no laboratório

Durante a apresentação, o grupo deverá:

- executar o programa;
- explicar brevemente como o problema foi decomposto;
- identificar no código:
 - uma variável comum;
 - um ponteiro;
 - o operador &;
 - o operador *;
 - um parâmetro passado normalmente por valor;
 - um parâmetro que recebe um endereço;
- explicar por que trocar(a, b) não permitiria modificar as variáveis originais;
- explicar por que trocar(&a, &b) permite modificá-las;
- executar pelo menos um dos cenários de referência.

Em seguida, o professor escolherá uma pequena alteração do programa.



O grupo terá aproximadamente **5 minutos** para realizá-la na frente da tela.

A alteração deverá ser feita sobre o código apresentado, **sem substituir o programa inteiro**, seguindo a mesma dinâmica do trabalho anterior.

Trabalho — Funções e Subprogramas em C.pdf